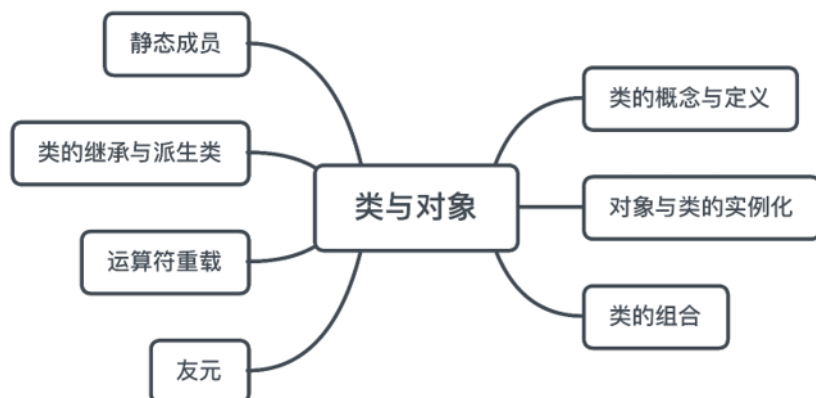




专题十一 类与对象

【内容预览】



【知识清单】

11.1、类

一、类的定义及其语法形式

“类”是一种用户自定义的数据类型，其实质是对具有共同特征的事物的抽象。类将数据和数据的处理方法封装在一起。声明一个类的语法形式为：

```
class 类名{  
    ...  
};
```

二、类的成员

类的成员包括数据成员和函数成员，用数据表示“类”的属性，用函数表示“类”的行为，功能或者对数据成员的操作。

1. 数据成员：

数据成员定义在类体中，语法形式为：

```
数据类型 成员名
```

2. 函数成员：

1) 定义形式：在“类”中函数成员有两种定义方法。第一种是直接在类体中完成函数体的实现，语法形式为：

```
返回值类型 成员函数名(参数表){
    函数体
};
```

第二种是先在类体中进行函数原型声明，再在类体外进行函数体实现。在类体外部定义成员函数时必须先在成员函数前给出所属的类名，语法形式为：

```
返回值类型 类名::成员函数名(参数表){
    函数体
}
```

注意：

类将数据与数据的处理函数封装在一起，因而在定义函数成员时，如果将类看成一个封闭的域，类的数据成员就相当于域中的全局变量，可以被类的成员函数直接使用和共享。

2)分类：类体中的特殊函数成员主要包括**构造函数**，**复制构造函数**和**析构函数**这三种函数。

构造函数：在定义对象的时候也同时可以对其赋初始值，这称为**对象的初始化**，而调用构造函数进行对象的初始化是 C++ 的标准方法。当程序实例化对象时，如果程序员没有定义构造函数，那么 C++ 编译器会提供一个默认的构造函数以供使用，但系统提供的构造函数没有参数，而且是空函数，什么赋值工作也不做；一旦程序员定义了构造函数，C++ 的编译器则不会自动生成默认的构造函数。如果程序员人为定义的函数中没有定义形式参数，则该构造函数就被称为默认构造函数。

构造函数的语法形式：

```
类名(参数表){
    函数体
};
```

注意：

1. 构造函数是类的 public 成员函数，**函数名与类名相同**。
2. 构造函数无函数返回类型说明。（**什么都不写，包括 void**）
3. 构造函数**右花括号后边要带一个分号**。
4. **一个类可以有多个构造函数**，系统会根据对象定义时的形式自动完成函数的重载，调用最匹配的构造函数完成对象的初始赋值。
5. **构造函数的功能是为对象的数据成员赋初始值**，所以构造函数的形式参数的数量和类型都与对象的数据成员数量和类型有关系。

例 11-1 定义一个时钟类

```
class Clock{
    int hp,mp,sp;
public:
    Clock(int h,int m,int s){
        hp=h,mp=m,sp=s;
    };//构造函数
    Clock(){
    };//默认构造函数
}
```

当使用语句 `Clock c1;` 实例化对象时，系统会调用默认构造函数；当使用语句 `Clock c2(1, 3, 5);` 实例化对象时，系统会调用含参数的构造函数。

复制构造函数：复制构造函数只有一个形式参数，就是对本类对象的引用，其功能是将形参对象的数据成员复制给当前正在创建的新对象的数据成员，完成新对象的初始化。作为构造函数的一员，除了参数形式必须是本类对象的引用外，复制构造函数具有构造函数的所有语法特性。

复制构造函数的语法形式：

```
类名（类名 &引用的对象名）{
    函数体
};
```

注意：

1. 复制构造函数是将形参对象的数据成员复制给当前正在创建的新对象的数据成员，所以在函数体中赋值时要注意是对象的使用，即赋值格式为**数据成员名=对象名.数据成员名**。
2. 对象和普通变量一样，同类对象之间可以互相赋值，而复制构造函数只是在对象创建时由系统自动调用完成新对象的初始化，所以形如“对象名1=对象名2”的赋值语句并不会使用复制构造函数。

析构函数：和普通变量一样，对象也有生命期。当一个对象的生命周期结束的时候，C++将会调用一个特殊的成员函数对对象进行善后工作，该函数称为析构函数。和构造函数一样，C++的编译器会给出一个默认的析构函数，但是如果程序员自己定义了一个析构函数，系统就不会生成默认的析构函数。

复制构造函数的语法形式：

```
~类名（）{
    函数体
};
```

注意：

1. 析构函数是类的 public 成员函数，**函数名和类名相同**，但是要在**前面加上一个~**。
2. 析构函数无函数返回类型说明。（**什么都不写，包括 void**）
3. 构造函数**右花括号后边要带一个分号**。
4. **一个类只有一个析构函数**，析构函数在对象生命周期结束时由系统自动调用执行。
5. 如果程序中有多个对象消亡，**析构函数的调用顺序和构造函数的顺序相反**。

11.2、对象

一、对象的定义及语法形式

“类”数据类型的变量称为“对象”，声明一个对象叫做对类的实例化或实例化对象。实例化对象的语法形式为：

```
类名 对象名;
```

或

```
类名 对象名(参数表);
```

二、对象的使用

使用对象实际上是外部使用对象的成员，语法形式为：

```
对象名.成员名
```

注意：使用对象实际上是通过调用对象的成员函数来完成程序的功能，这种调用属于类外访问，受到访问限定符的约束，只能使用 public 属性的成员。

三、this 指针

在 C++ 中，每一个对象都能通过 this 指针来访问自己的地址。this 指针是**所有成员函数的隐含参数**。因此，在成员函数内部，它可以用来指向调用对象。this 指针调用对象的一般形式为：

`this->成员名;`

例如：

```
class Box {
    double length, breadth, height;
public:
    Box() {
    }
    double getVolume(){
        .....
    }
    int compare(Box box) {
        return this->getVolume() > box.getVolume();
    }
};
```

通过 this 指针调用自身的 getVolumn()函数，并与传参进来的对象的 getVolumn()函数的值做比较。在实际使用中，return this->getVolume() > box.getVolume();与 return getVolumn() > box.getVolumn();并无差别。

this 指针是一个常量指针，它始终指向本对象的地址。

11.3、类的组合

在面向对象的程序设计方法中，复杂对象也可以通过组合多个简单对象来实现，其方法就是将简单对象作为复杂对象的数据成员。

例 11-2 求矩形的宽和高

```
#include <iostream>
using namespace std;
class Point{
    double x,y;
public:
    Point(double xx=0, double yy=0){
        x = xx;
        y = yy;
    }
    double getX(){
        return x;
    }
    double getY(){
        return y;
    }
};
```

```

};                                     //定义 Point 类
class Rect{
    Point p1, p2;
public:
    Rect(Point a, Point b){
        p1 = a, p2 = b;
    }
    int RectHeight(){
        return p1.getY() > p2.getY() ? p1.getY() - p2.getY() : p2.getY() - p1.getY();
    }                                     //定义矩形的高
    int RectWidth(){
        return p1.getX() > p2.getX() ? p1.getX() - p2.getX() : p2.getX() - p1.getX();
    }                                     //计算矩形的宽
};                                     //定义 Rect 类
int main(){
    Point p1(1,1), p2(5,5);
    Rect crect(p1,p2);
    cout << "矩形的高=" << crect.RectHeight() << endl;
    cout << "矩形的宽=" << crect.RectWidth() << endl;
    return 0;
}

```

主要有以下几个注意事项:

注意:

1. 组合类定义时, 将简单对象声明为组合类的数据成员。
2. 组合类的构造函数定义时, 对象成员的初始化要使用初始化列表。
3. 对象成员的成员不是组合类的成员。

11.4、友元

友元的意义及其语法形式:“封装”是类的基本特性, 在这一特性约束下, 外部语句和外部函数对类的私有成员没有直接访问权限。友元则是 C++提供的突破这层限制的机制, 声明友元的一般形式为:

`friend` 类名或函数名

例 11-3

```

#include<iostream>
using namespace std;
class A{
    int a, b;
    friend int add(A &);           //声明函数 add()是类 A 的友元函数
public:
    A(){
        a = 0, b = 0;
    };
    A(int x, int y){
        a = x, b = y;
    };
}

```

```
};
int add(A &a){
    return a.a + a.b;           //可以直接访问类 A 的私有成员
}
int main(){
    A a(1, 2);
    cout << add(a) << endl;
    return 0;
}
```

在例 11-2 中，如果在 Point 类声明 Rect 类是其友元类，则在 Rect 类中可直接访问 Point 类的私有成员，如下：

例 11-4

```
#include <iostream>
using namespace std;
class Point{
    double x,y;
    friend class Rect;           //声明 Rect 类为友元类
public:
    Point(double xx=0, double yy=0){
        x = xx;
        y = yy;
    }
};                               //定义 Point 类
class Rect{
    Point p1, p2;
public:
    Rect(Point a, Point b){
        p1 = a, p2 = b;
    }
    int RectHeight(){
        return p1.y > p2.y ? p1.y - p2.y : p2.y - p1.y;
    }                             //定义矩形的高
    int RectWidth(){
        return p1.x > p2.x ? p1.x - p2.x : p2.x - p1.x;
    }                             //计算矩形的宽
};                               //定义 Rect 类
int main(){
    Point p1(1,1), p2(5,5);
    Rect crect(p1,p2);
    cout << "矩形的高=" << crect.RectHeight() << endl;
    cout << "矩形的宽=" << crect.RectWidth() << endl;
    return 0;
}
```

注意：1. 友元关系是单向的，且不能传递。即若 A 是 B 的友元，则 A 可以访问 B 的私有成员，而 B 不可以访问 A 的私有成员；若 B 是 C 的友元，A 不可以访问 C 的私有成员。

2. 为了确保数据的完整性及数据封装与隐藏的原则，尽量不使用或少使用友元。

11.5、运算符重载

一、运算符重载的意义

变量的数据类型不仅决定了变量的存储形式、占用内存的大小，还限制了变量能完成的运算类型。由于无法预知用户定义的类的细节，运算符（赋值运算符除外）的运算对象默认只能是基本数据类型的变量。如定义如下一个复数类 `Complex` 和 `Complex` 类的对象 `c1,c2,c3`：

```
class Complex{
    double real, image;
public:
    Complex(double r = 0.0, double i = 0.0){
        real = r, image = i;
    };
};

Complex c1(1.3, 2.5), c2(1.5, 3.7), c3;
```

如果使用表达式 `c3 = c1 + c2`;则会报错。要实现两个复数相加，可以在 `Complex` 类里定义一个 `add()`函数来实现；也可以通过运算符重载来实现。

例 11-5 对运算符“+”进行重载

```
#include <iostream>
using namespace std;
class Complex{
public:
    double real, image;
    Complex(int r, int i){
        real = r, image = i;
    }
    Complex(){
        real = image = 0; }
    void show(){
        cout<<real<<"+"<<image<<"i"<<endl;
    }
    Complex operator+(Complex c){
        Complex temp;
        temp.real = this->real + c.real;
        temp.image = this->image + c.image;
        return temp;
    }
};

int main(){
    Complex c1(4,5),c2(6,7),c3;
    c1.show();
    c2.show();
    c3 = c1 + c2;
    c3.show();
    return 0; }
```

注：除了成员访问运算符`.`、成员指针访问运算符`* , ->`、域运算符`::`、长度运算符`sizeof`、条件运算符`?:`、预处理符号`#`之外的运算符均可重载。

11.6、类的继承

一、继承

继承允许我们依据另一个类来定义一个类，以达到重用代码功能和提高执行时间的效果。这个已有的类称为**基类**，新定义的类称为**派生类**；同时，基类称为派生类的**父类**，派生类称为基类的**子类**。

继承的一般形式为：

```
class 派生类名:继承类型 基类名{
    .....
}
```

表 13-1:继承类型

继承类型	作用
公有继承（public）	当一个类派生自公有基类时，基类的公有成员也是派生类的公有成员，基类的保护成员也是派生类的保护成员，基类的私有成员不能被派生类访问，但是可以通过调用基类的公有和保护成员来访问。
保护继承（protected）	当一个类派生自保护基类时，基类的公有和保护成员将成为派生类的保护成员。
私有继承（private）	当一个类派生自私有基类时，基类的公有和保护成员将成为派生类的私有成员。

注：一般不使用 protected 和 private 继承。

例 11-6

```
#include <iostream>
using namespace std;
class Shape{
public:
    void setWidth(int w){
        width = w;}
    void setHeight(int h){
        height = h;}
protected:
    int width;
    int height;
};
class Rectangle : public Shape{
public:
    int getArea(){
        return (width * height); //width 和 height 均继承于 Shape 类
    }
};
int main(){
    Rectangle Rect;
    Rect.setWidth(5);           //setWidth()和 setHeight()均继承于 Shape 类
    Rect.setHeight(7);
    cout << "Total area: " << Rect.getArea() << endl;
    return 0;
}
```


二、多继承

一个类可以派生自多个类，称为多继承。多继承的子类可以继承多个父类的特性。
多继承的一般形式为：

```
class 派生类名:继承方式 基类名, 继承方式 基类名, .....{
    .....
};
```

例 11-7

```
#include <iostream>
using namespace std;
class Shape{
public:
    void setWidth(int w){
        width = w;
    }
    void setHeight(int h){
        height = h;
    }
protected:
    int width;
    int height;
};
class PaintCost{
public:
    int getCost(int area){
        return area * 70;
    }
};
class Rectangle: public Shape, public PaintCost{
public:
    int getArea(){
        return (width * height);
    }
};
int main(void){
    Rectangle Rect;
    int area;
    Rect.setWidth(5);
    Rect.setHeight(7);
    area = Rect.getArea();
    cout << "Total area: " << Rect.getArea() << endl;
    cout << "Total paint cost: $" << Rect.getCost(area) << endl;
    return 0; }
```

11.7、静态成员

一、静态数据成员

一般来说，由类实例化出的对象都拥有独立的数据成员，每实例化一个对象，系统都会为它的数据成员分配独立的内存空间。而类的静态成员则不然，它是属于整个类的，被所有由这个类实例化出的对象所共享。定义静态数据成员的一般形式为：

`static 变量类型 变量名;`

我们不能把静态成员的初始化放置在类的定义中，但是可以在类的外部通过使用范围解析运算符 `::` 来重新声明静态变量从而对它进行初始化。如果不存在其它的初始化语句，则在创建第一个对象时，所有的静态数据都会被初始化为零。例如：

```
class Box{
    public:
        static int objCount;
        .....
};
.....
int Box::objCount = 0;
```

二、静态函数成员

与静态数据成员类似，如果把函数成员也声明为静态的，就可以把函数与类的任何特定对象独立开来。静态成员函数即使在类对象不存在的情况下也能被调用，只需要在函数名称前加上类名加域运算符 `::` 就可以访问。静态成员函数只能访问静态成员数据、其他静态成员函数和类外部的其他函数。例如：

```
class Box{
    public:
        static int objCount;
        .....
        static int getCount(){
            return objCount;
        }
        static void initCount(){
            objCount = 0
        }
};
.....
int main(){
    .....
    Box::initCount();
    int count = Box::getCount();
    .....
}
```

【解题技巧】

例11-8 关于运算符重载的不正确的描述是（ ）

- A. 运算符重载函数是友元函数
- B. 体现了程序设计的多态性
- C. 不能增加新的运算符
- D. 使运算符能对对象操作

正解: D

分析: A 运算符重载函数是成员函数, 因此不是友元函数。

例 11-9 从程序片断: `student zhangsan("张三","M",22); zhangsan.id("2005131000");` 可判断 id 是一个 ()

- A. 私有数据成员 B. 私有成员函数 C. 公有数据成员 D. 公有成员函数

正解: D

分析: id 需要接收参数, 因此是函数成员; 在类外可以直接访问, 因此是公有成员。

【精选习题】

答案 P119

基础篇

- 下列关于构造函数的论述中, 不正确的是 ()
 - 构造函数的函数名与类名相同
 - 构造函数可以设置默认参数
 - 构造函数的返回类型缺省为 int 型
 - 构造函数可以重载有以下程序
- 对类的构造函数和析构函数描述正确的是 ()
 - 构造函数可以重载, 析构函数也可以重载
 - 构造函数不能重载, 析构函数也可以重载
 - 构造函数可以重载, 析构函数也不能重载
 - 构造函数不能重载, 析构函数也不能重载
- 假定 AB 为一个类, 则执行“`AB r1=r2;`”语句时将自动调用该类的 ()
 - 带参构造函数
 - 无参构造函数
 - 赋值重载函数
 - 复制构造函数
- 若有一下程序

```
#include<iostream>
using namespace std;
class A{
    private: int a;
    public: A(int i)  {
        a=i;
    }
    void disp() {
        cout<<a<<" ";
    }
};
class B{
    private: int b;
    public: B(int j) {
        b=j;
    }
    void disp() {
        cout<<b<<" ";
    }
};
```

```
class C:public B, public A{
private:
    int c;
public:
    C(int k):A(k-2),B(k+2) {
        c=k;
    }
    void disp() {
        A::disp();
        B::disp();
        cout<<c<<endl;
    }
};

int main(){
    C obj(10);
    obj.disp();
    return 0;
}
```

程序执行后的输出结果是 ()

- A. 10, 10, 10 B. 10, 12, 14 C. 8, 10, 12 D. 8, 12, 10

提高篇

- 下面对静态数据成员的描述中，正确的是 ()
 - 静态数据成员不可以被类的对象调用
 - 静态数据成员可以在类体内进行初始化
 - 静态数据成员不能受 `private` 控制符的作用
 - 静态数据成员可以直接用类名调用
- 在一个派生类对象介绍其生命周期时 ()
 - 先调用派生类的析构函数后调用基类的析构函数
 - 先调用基类的析构函数后调用派生类的析构函数
 - 如果基类没有定义析构函数，则只调用派生类的析构函数
 - 如果派生类没有定义析构函数，则只调用基类的析构函数
- 定义一个人员类 `Person`，包含一个字符数组 `name[12]`，一个表示助学金 `allowance` (`double`) 的数据成员，二个分别获取姓名和助学金的成员函数 `getName` 和 `getAllowance`，还包含构造函数、复制构造函数和设置姓名和助学金的函数 `set`。请补充完成 `Person` 类。输出结果见图

```
-----*/
#include <iostream>
#include <string>
using namespace std;
class Person{
    char name[12];
    double allowance;
public:
    Person(char *n, double a);
    Person(Person &p);
    char *getName(void);
    double getAllowance(void);
```



```
        void set(char *n, double a);
    };
    /*******Program*****/

    /******* End *****/
    void display(Person p)
    {
        cout<<"姓名="<<p.getName()<<"\t";
        cout<<"助学金="<<p.getAllowance()<<endl;
    }
    int main()
    {
        Person s1("王大海", 210);
        Person s2=s1;
        s2.set("张大山", 200);
        display(s1);
        display(s2);
        return 0;
    }
```